Module 4: Account Abstraction

Smart Accounts & Programmatic Control

What is Account Abstraction?

Traditional Accounts

- **User Accounts**: Controlled by private keys
- **Program Accounts**: Immutable executable code
- **PDAs**: Deterministically derived addresses

Account Abstraction

- **Smart Accounts**: Programmatic control over account behavior
- **Validation Rules**: Custom spending limits and access controls
- **Enhanced Security**: Multi-signature and whitelist features

Architecture Overview

Validation Rules Structure

Rules Object Configuration

```
{
  "maxDailySpend": 1000000,          // Daily limit in lamports
  "allowedPrograms": [               // Program ID whitelist
    "TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA",
    "11111111111111111111111111111112"
  ],
  "requiredSigners": 2               // Multi-sig requirement
}
```

Rule Types

- **Daily Spending Limits**: Redis-tracked budgets
- **Program Whitelisting**: Allowed program IDs only
- **Multi-signature Support**: Required number of signers

API Endpoints

Smart Account Management

- `POST /smart-accounts` - Create new smart account
- `GET /smart-accounts/:address` - Get smart account details
- `POST /smart-accounts/:address/validate` - Validate transaction against rules

Validation Logic

```
validateTransaction(tx: Transaction): {valid: boolean, reason?: string} {
  // Check account status
  if (account.status !== 'ACTIVE') {
    return {valid: false, reason: 'Account not active'};
  }

  // Check daily spend limit
  const dailySpent = await redis.get(`smart-account:${address}:spent:${today}`);
  if (dailySpent + tx.amount > rules.maxDailySpend) {
    return {valid: false, reason: 'Daily spend limit exceeded'};
  }
```

Smart Account Features

Implemented Capabilities

- **Daily Spending Limits**: Redis-tracked spending budgets
- **Program Whitelisting**: Restrict interactions to approved programs
- **Multi-signature Support**: Require multiple signers for transactions
- **Account Status Control**: ACTIVE/FROZEN/DISABLED account states
- **Event-driven Architecture**: Kafka integration for monitoring

Security Benefits

- **Reduced Attack Surface**: Limited program interactions
- **Spending Controls**: Prevent unauthorized large transfers
- **Account Recovery**: Multi-sig for lost key scenarios
- **Audit Trail**: Complete transaction history and validation logs

PDA Integration

Program Derived Addresses

```
// Mock implementation for development
const mockSmartAccountAddress = `smart-${ownerAddress}-${timestamp}`;

// Future: Real PDA derivation
const [smartAccountAddress, bump] = await PublicKey.findProgramAddress(
  [
    Buffer.from('smart-account'),
    new PublicKey(ownerAddress).toBuffer(),
    Buffer.from(timestamp.toString())
  ],
  programId
);
```

PDA Benefits

- **Deterministic**: Same inputs always generate same address

Implementation Status

✅ Completed Features

- Smart account entity with validation rules
- Redis caching for rules and spending tracking
- Kafka event streaming for account creation
- API endpoints for CRUD operations
- Transaction validation logic

🚧 Future Enhancements

- Real PDA implementation with Solana Web3.js
- Multi-signature wallet integration
- Gasless transaction support
- Account recovery mechanisms

Key Takeaways

Account Abstraction Benefits

- **Enhanced Security**: Programmatic spending controls and whitelisting
- **User Experience**: Gasless transactions and account recovery
- **Developer Flexibility**: Custom validation rules and business logic
- **Scalability**: Off-chain validation with on-chain enforcement

SVM Advantages

- **PDAs**: Trustless deterministic address generation
- **Parallel Processing**: High-throughput validation and execution
- **Low Fees**: Cost-effective account management
- **Composability**: Smart accounts can interact with any program